

多輪引導式漏斗框架(中小型房車銷售)實作計畫

1. 核心模組概覽

1.1 對話狀態管理模組 (Dialogue State Tracking, DST)

- **目的**: 維護最新的對話上下文、銷售漏斗階段、已收集的槽位值, 以及推薦記錄。
- **主要欄位**:
 - ``chat_history``: 逐輪對話訊息。
 - ``filled_slots``: 槽位名稱 → 值的字典(例如 ``budget``、``seat_count``、``usage``)。
 - ``recommendations``: 已建議的產品 ID/名稱列表。
- **技術**: ``Python TypedDict``、``LangGraph Stateful Graph``、``LangChain Memory``。
- **潛在問題**: 狀態遺失、槽位結構不完整、槽位鍵與定義不一致。

1.2 對話控制模組 (Dialogue Manager, DM)

- **目的**: 根據當前對話狀態決定下一步系統行動。
- **主要功能**:
 - 對話流程路由(``Think`` 步驟)。
 - 執行商業漏斗邏輯。
- **技術**: ``LangGraph Conditional Edges``、``LangChain PromptTemplate``、支援結構化輸出的 LLM。
- **潛在問題**: 用戶意圖判斷錯誤、對話偏離漏斗路徑。

1.3 槽位填充引擎 (Slot Filling Engine)

- **目的**: 蒐集購車關鍵需求(例如預算、座位數、燃料類型、品牌偏好)。
- **主要特色**: 必填/選填標記、多值槽位、澄清問題模板。
- **技術**: 基於 LLM 的實體抽取、提示鏈、LangChain 工具調用。
- **潛在問題**: 用戶輸入模糊、同義詞對應、缺乏必要澄清。

1.4 推薦引擎 (Recommendation Engine)

- **目的**: 篩選車款資料庫並生成有說服力的推薦。
- **實作方法**:
 1. 基礎版: 程式過濾。

2. 進階版:基於 RAG 的語義檢索 + LLM 綜合生成。

- **技術**: `Pandas`、`FAISS`/`Chroma`、`LangChain RetrievalQA`。
- **潛在問題**: 相關性排序不佳、生成虛構規格。

1.5 提示鏈模組 ("Think, Then Act")

- **目的**: 將策略決策(Think)與面向用戶的回應生成(Act)拆分, 降低 LLM 偏離風險。
- **技術**: `LangChain ChatPromptTemplate`、結構化輸出(Pydantic 模型)。
- **潛在問題**: 模板變數錯誤、提示過於寬鬆導致偏離目標。

1.6 車款知識庫 (Vehicle Knowledge Base)

- **目的**: 儲存中小型房車的詳細規格, 供搜尋與檢索。
- **資料結構**: CSV, 欄位包含品牌、型號、座位數、燃料類型、價格區間、功能特色等。
- **技術**: `Pandas` 載入 CSV、向量資料庫做 RAG。
- **潛在問題**: 資料更新不及時、不同品牌欄位結構不一致。

1.7 長期記憶與個人化模組 (Long-Term Memory & Personalization)

- **目的**: 跨會話保存用戶資料, 提供個性化體驗。
- **方法**: 以 `user\_id` 為鍵, 將 DialogueState 存於 Redis 或 PostgreSQL。
- **技術**: `SQLAlchemy`、`Redis`、LangGraph Checkpointing。
- **潛在問題**: 隱私與合規問題、用戶身份辨識可靠性。

1.8 錯誤處理與對話修復模組 (Error Handling & Conversational Repair)

- **目的**: 處理離題問題、誤解、或用戶情緒負面情況。
- **策略**:
 - 重述問題。
 - 提供多選選項。
 - 支援用戶中斷與切換需求。
 - 必要時升級至人工客服。
- **技術**: 情感分析、LangGraph 後備路由。
- **潛在問題**: 過度打斷用戶、修復策略顯得機械。

2. 技術棧

- **核心**: Python、LangChain、LangGraph
- **LLM 供應商**: OpenAI / Anthropic
- **資料處理**: Pandas
- **向量資料庫**: FAISS / Chroma
- **資料持久化**: Redis / PostgreSQL
- **結構化輸出驗證**: Pydantic

3. 潛在挑戰

1. **LLM 路由錯誤** — 可能導致漏掉必要的漏斗步驟。
2. **資料時效性** — 車款知識庫需定期更新。
3. **提示穩定性** — 在銷售語氣與自然對話之間找到平衡。
4. **同義詞與模糊解析** — 確保槽位填充理解不同說法。
5. **效能** — RAG 與 LLM 請求可能導致延遲, 需快取策略。

4. 實作階段

1. **資料準備**
 - 定義中小型房車銷售的槽位結構。
 - 建立初始車款知識庫 CSV。
2. **核心框架建立**
 - 實作 `DialogueState` 結構。
 - 初始化 LangGraph 狀態化流程。
3. **路由與提示鏈**

- 建立 `route\_action` 函數(先程式版, 後 LLM 版)。
- 實作 `Think` 與 `Act` 提示模板。

4. **槽位填充**

- 實作槽位抽取邏輯。
- 建立澄清問題生成。

5. **推薦邏輯**

- 實作基礎篩選。
- 整合 RAG 管線。

6. **錯誤處理**

- 增加離題、修復與升級流程。

7. **持久化層**

- 實作跨會話狀態保存與讀取。

8. **測試與優化**

- 模擬多種用戶路徑。
- 調整提示、槽位結構與檢索品質。
